



Mastering Oracle Cloud Infrastructure (OCI) Architect & DevOps Professional (Hybrid Curriculum)

FOCUS: Real-Time Scenarios, Migration & Automation

This program is designed to prepare students not only for OCI certifications but for real production work from day one. By the end of the course, students/professional will:

- Design high-level and low-level architectures for OCI workloads (HLD/LLD).
- Implement infrastructure using Terraform and Git-driven workflows (Infrastructure DevOps).
- Troubleshoot networking, compute, storage, and security issues in real-world-like scenarios.
- Execute end-to-end migration and disaster recovery flows across regions and tenancies.

Target Graduate Profile: "Hybrid Cloud Engineer / Architect + DevOps" who can design, automate, and support OCI environments end-to-end.

Module 1 – The Engineer's Toolkit (Day 0 Setup)

Objective: Standardize each student's workstation to match enterprise DevOps practices and ensure they can work with Git, VS Code, and OCI from day one.

Topics

Developer Tools Overview

- VS Code installation, extensions for Terraform, and SSH remote editing.
- PuTTY, PuTTYgen, WinSCP for SSH and file transfer.

Version Control & Collaboration

- Git installation and configuration (user.name, user.email, SSH keys).
- git clone, git status, git add, git commit, git push, branching basics.
- GitHub account creation and repository setup (code + Wiki).

AI-Augmented Development

- GitHub Copilot / inline chat setup in VS Code for Terraform and shell.

Linux Essentials for Cloud

- File system structure: /, /home, /var, /var/log, /etc, /tmp.
- Core commands: ls, cd, pwd, cat, tail, grep, mkdir, cp, mv, rm, df -h, du -sh.
- Process and service troubleshooting: ps -ef, top, systemctl, service.
- Network troubleshooting tools: ping, telnet, netstat, firewall-cmd.

Real-World Scenario

"New Joiner – Workstation Readiness"

You have joined an OCI team; your first ticket is to set up your workstation, configure Git, join the team repo, and prove you can SSH into a test VM.

Hands-on Labs (Module 1)

Lab 1: Install VS Code and configure essential extensions (Terraform,SSH).

Lab 2: Create GitHub account, create a repository, enable Wiki, and push first README.

Lab 3: Enable GitHub Copilot / VS Code AI assistance and generate a sample Terraform snippet.

Lab 4: Install PuTTY, PuTTYgen, WinSCP and test file transfer between local and a test VM (dummy or future lab VM).

Lab 5: Explore Linux file system on a sample OCI compute instance (df -h, du -sh, /var/log usage).

Lab 6: Practice core Linux commands and navigation; document 10 frequently used commands in your Wiki.

Lab 7: Work with the vi editor: open, edit, search, save, and quit; create a "notes.sh" script file.

Module 2 – Cloud Foundations & Tenancy Design (Identity & Governance)

Objective: Understand OCI global architecture and design a secure tenancy structure using compartments, IAM, tags, and budgets.

Topics

OCI Global & Physical Architecture

- Tenancy, Regions, Availability Domains, Fault Domains.
- Difference between on-premises vs. cloud vs. multi-cloud.

Logical Isolation & Governance

- Compartment design for prod, non-prod, shared services, network, and DR.
- Tagging strategy (environment, cost center, project).
- Budgets and cost analysis.

Identity & Access Management

- Users, groups, policies, dynamic groups.
- Policy language: subject, verb, resource-type, conditions.
- Sample "practice group" policies for instances, network, storage, vault, monitoring, etc.

Real-World Scenario

"Landing Zone for HR & Finance"

Design a landing zone where HR and Finance workloads are strictly separated at the compartment level with clear policies for admins, support, and read-only users.

Hands-on Labs (Module 2)

Lab 8: Create OCI tenancy account and perform region selection exercise, documenting rationale in Wiki.

Lab 9: Design and create a multi-level compartment structure (root → app/network/storage/DR).

Lab 10: Create users and groups for Admin, Dev, Ops, and ReadOnly roles.

Lab 11: Write IAM policies to grant manage/use/read rights to specific groups for instance-family, volume-family, object-family, keys, vaults, metrics, and policies in a practice compartment.

Lab 12: Configure tags (environment, owner, application) and apply them to key resources.

Lab 13: Create a budget and cost alert thresholds for a specific compartment.

Lab 14: Use OCI Cost Estimator to estimate monthly cost of a small 3 Tier environment.

Module 3 – Core Networking (VCN Fundamentals)

Objective: Master IP planning, VCN design, subnets, security lists, and basic connectivity for simple architectures.

Topics

IP Addressing & Subnetting

- RFC 1918 ranges (10.x, 172.16.x, 192.168.x) and /24, /25, /28 sizing.

VCN & Subnet Design

- VCN address space planning and non-overlapping CIDRs.
- Public vs. private subnets and their use cases.

Basic Firewalling

- Security Lists: ingress/egress rules, default deny mindset.
- Self-referencing rules, ICMP allow rules.

Basic Troubleshooting

- Ping and telnet for port reachability.

Real-World Scenario

"Secure Web + DB Application"

Build a simple VCN with one public subnet for web and one private subnet for DB, ensuring only HTTP/HTTPS access from internet to web tier and DB only accessible from web subnet.

Hands-on Labs (Module 3)

Lab 15: CIDR planning and IP subnetting calculations for a small environment.

Lab 16: Create a VCN and subnets using the console wizard; document VCN design in draw.io later.

Lab 17: Manually create a security list and associate it with a subnet.

Lab 18: Add specific ingress/egress rules (e.g., allow SSH from your public IP, allow HTTP/HTTPS from 0.0.0.0/0, deny others).

Lab 19: Design and implement a security list rule that allows traffic from one CIDR while excluding another (complex firewall rule).

Lab 20: Replace 0.0.0.0/0 with your own IP in security lists to harden remote management access.

Lab 21: Use telnet and ping to test port reachability between two instances in the same subnet; document troubleshooting steps.

Module 4 – Compute & Storage Operations (IaaS)

Objective: Provision, secure, and operate OCI compute instances and storage (block, boot, object), including troubleshooting and performance tuning.

Topics

Compute Fundamentals

- Shapes, OCPU vs vCPU, Flex shapes.
- SSH key-based access, heterogeneous vs homogeneous key pairs.
- Enabling password-based access for selected scenarios.
- Using Serial Console to fix SSH issues.

Block & Boot Volumes

- Creating block volumes, attaching/detaching, backup policies.
- Resizing volumes and expanding file systems online.
- Detaching boot volumes and attaching to recovery instances.

Object Storage

- Buckets, standard vs archive, lifecycle basics.

Real-World Scenario

"Disk Full – Production Outage"

An application server is out of disk space; extend storage, resize the filesystem online, and ensure application recovers without reboot.

Hands-on Labs (Module 4)

Lab 22: Create your first compute instance in a public subnet using key-based SSH; verify connectivity via PuTTY.

Lab 23: Configure password-based access for controlled admin usage, then revert to hardened key-only access.

Lab 24: Create and attach a block volume; create a Linux filesystem and mount it.

Lab 25: Implement a boot/data disk backup policy for compute instances.

Lab 26: Resize a block volume and extend the filesystem online on Linux (xfs or ext4). Document commands used.

Lab 27: Detach a boot volume from a faulty instance and attach it to a recovery instance; fix configuration and reattach.

Lab 28: Create a custom image from a hardened application VM.

Lab 29: Provision a new application instance from a custom image in a private subnet and validate access through a jump/bastion host.

Module 5 – Advanced Networking (Hub & Spoke, DRG, Gateways)

Objective: Design and implement hub–spoke network topologies with DRG, NAT, Service Gateway, and bastion patterns.

Topics

Network Topologies

- Mesh vs hub-spoke networks.
- Asymmetric routing concept and impact on troubleshooting.

Gateways

- Internet Gateway, NAT Gateway, Service Gateway.
- DRG attachments and route tables for hub-spoke interconnect.

Bastion & Private Access Patterns

- Designing bastion servers in a hub VCN.

Network Visibility

- Network Visualizer and Network Path Analyzer for path tracing and debugging.

Real-World Scenario

"Shared Services Hub for 3 Applications"

Build a central hub VCN with firewall and monitoring tools; connect multiple spoke VCNs, ensuring spokes reach internet via the hub only.

Hands-on Labs (Module 5)

Lab 30: Create an Internet Gateway and attach route tables for outbound internet access.

Lab 31: Create a bastion server in a public subnet and use it to reach a private-instance in the same VCN.

Lab 32: Build a hub VCN with manual public subnet, security lists, IGW, and routes; deploy a hub instance.

Lab 33: Create a DRG and attach hub and spoke VCNs.

Lab 34: Configure routes and security lists for hub–spoke connectivity (hub public subnet to spoke private subnet).

Lab 35: Validate inter-VCN connectivity between hub and spoke using ping and application ports.

Lab 36: Use Network Visualizer to view the entire topology.

Lab 37: Use Network Path Analyzer to validate forwarding paths and identify misconfigurations.

Lab 38: Configure Service Gateway for private access to Object Storage from a private subnet in a spoke VCN.

Lab 39: Configure NAT Gateway for one-way internet access from private instances.

Module 6 – Infrastructure as Code & DevOps (Terraform on OCI)

Objective: Automate OCI infrastructure using Terraform, version control, and reusable modules.

Topics

Terraform Fundamentals

- Provider configuration for OCI.
- Resources, data sources, variables, outputs.
- terraform init, plan, apply, destroy, validate.

State Management

- Local state vs remote state in Object Storage.

Code Quality & Structure

- Local variables, reusable modules for VCN, subnets, compute.
- Safeguarding provider files with environment variables.

Real-World Scenario

"ClickOps is Banned"

All new VCNs and compute instances must be created using Terraform; your change must be reviewed and approved in Git before apply.

Hands-on Labs (Module 6)

Lab 40: Configure VS Code for Terraform, install Terraform CLI, and validate terraform -v output.

Lab 41: Initialize a Git repo for OCI Terraform code and push to GitHub.

Lab 42: Create provider.tf and variables.tf for OCI tenancy and region.

Lab 43: Write Terraform code to provision a single compute instance; run init, plan, apply.

Lab 44: Create Terraform resources for a basic spoke VCN (VCN, subnets, security lists).

Lab 45: Introduce local variables and outputs to clean up code and expose IP addresses and OCIDs.

Lab 46: Modularize VCN and subnet definitions into reusable modules; deploy multiple spokes using one module.

Lab 47: Implement environment variable–based credentials instead of hardcoded provider values.

Lab 48: Practice basic code review: open a pull request, add comments on Terraform code, and merge after review.

Module 7 – Security, Encryption & Monitoring

Objective: Harden OCI environments using Vault, KMS, key rotation, security lists/NSGs, notifications, and metric-based alerting.

Topics

Data Security & Encryption

- OCI Vault, keys (MEK, DEK), BYOK.
- Encrypting boot and block volumes and object storage with customer-managed keys.
- Key rotation and compliance requirements.

Network & OS Hardening

- Security list vs NSG usage patterns.
- Basic OS-level hardening: firewall, logging, monitoring packages (conceptual).

Monitoring & Alerts

- OCI metrics for CPU, memory, storage, network.
- Alarms and notification topics (ONS).

Real-World Scenario

"Banking Compliance – Customer-Managed Keys"

A banking customer requires all backups and storage to use customer-managed keys with regular rotation.

Hands-on Labs (Module 7)

Lab 49: Create a Vault and generate master keys; document key metadata.

Lab 50: Configure data-at-rest encryption for a bucket using customer-managed keys.

Lab 51: Rotate keys in Vault and verify impacts on encrypted resources.

Lab 52: Create object storage buckets and test upload/download using CLI from a compute VM.

Lab 53: Create an ONS topic and subscription; send a test notification.

Lab 54: Configure alarms on instance CPU, memory metric (via OS mgmt or script), and storage utilization.

Lab 55: Troubleshoot "Application is down" scenario using metrics, logs, and network checks (step-by-step documented runbook).

Module 8 – Application Architecture (2 Tier & 3 Tier on OCI)

Objective: Design and deploy 2- tier and 3- tier applications using OCI compute, Autonomous Database, and load balancers.

Topics

2- Tier Architecture

- App + DB pattern in single region.
- Converting public-facing app to private app behind bastion/NAT.

3- Tier Architecture

- Web, app, and DB tiers with security and subnet separation.
- Load balancer use cases.

Autonomous Database Basics

- Creating ADB, wallet, network access.

Real-World Scenario

"Greenfield Application Deployment"

Deploy a new 3- tier app with public web, private app, and private DB tiers, using Autonomous DB and a regional load balancer.

Hands-on Labs (Module 8)

Lab 56: Build network (VCN, subnets, security lists) for a simple 2- tier application using Terraform.

Lab 57: Create an application server in an app subnet and expose a sample web application to the internet.

Lab 58: Create an Autonomous Database instance for the app.

Lab 59: Connect from application server to ADB using wallet; create and query sample tables.

Lab 60: Implement application logic to read/write data to Autonomous DB (simple CRUD demo).

Lab 61: Convert public application instance into a private-only instance; test access via bastion and/or load balancer.

Lab 62: Design a 3- tier architecture diagram and implement web/app/DB subnets with appropriate security lists.

Lab 63: Deploy a 3- tier application stack and validate end-to-end functionality.

Module 9 – Migration & Disaster Recovery

Objective: Implement migrations within OCI and across clouds, and design DR strategies (hot-hot, hot-warm, hot-cold).

Topics

Migration Strategies

- Region-to-region migrations.
- Inter-tenancy migration patterns.
- High-level view of inter-cloud migration (OCI to Azure).

DR Concepts

- RTO, RPO, hot-hot, hot-warm, hot-cold.
- Boot volume replication, object storage replication.

Real-World Scenario

"Region Failure Simulation"

Primary region fails; bring up application stack in another region using replicated boot volumes, DB backups, and Terraform definitions.

Hands-on Labs (Module 9)

Lab 64: Document end-to-end migration steps for a sample application (discovery → design → cutover).

Lab 65: Implement DR by backing up a compute instance (boot + data) and restoring it in the same region.

Lab 66: Perform migration from one OCI region to another for a simple application stack.

Lab 67: Perform an inter-tenancy migration in the same cloud (export/import-based flow).

Lab 68: Basic concept of an inter-cloud migration plan (OCI to Azure) focusing on network, security, and data flows (concept & documentation only).

Module 10 – Cloud Shell, CLI & Automation Hooks

Objective: Use OCI Cloud Shell, CLI, and basic scripting to manage resources and integrate with DevOps workflows.

Topics

Cloud Shell

- Access, basic commands, integration with Tenancy.

OCI CLI

- Installation, configuration, and version validation.
- Common commands for compute, object storage, and networking.

Automation Hooks

- Using CLI and shell scripts for repetitive tasks (volume attach, backup trigger, etc.).

Hands-on Labs (Module 10)

Lab 69: Explore OCI Cloud Shell; list compartments, VCNs, and instances.

Lab 70: Install OCI CLI on a Linux VM; configure profile with API keys and validate with `oci --version`.

Lab 71: Use OCI CLI to upload and download objects to a bucket from a compute instance.

Lab 72: Write a simple shell script that checks block volume usage and sends a notification when threshold exceeds (pseudo integration to alarm or log).

Module 11 – Career Launchpad (HLD/LLD, Resume, Interviews)

Objective: Translate technical work into documentation, resume content, and interview answers.

Topics

Documentation Skills

- Writing HLD vs LLD for an OCI workload.
- Using draw.io for architecture diagrams.

Resume & Job Portals

- Highlighting migration, hub-spoke, Terraform, and DR work.

Interview Preparation

- L1 and L2 interview patterns.

- Scenario-based Q&A: access issues, encryption, policies, DR, hub-spoke, transit routing, object storage security.

Real-World Scenario

"Design Defense – Present to CTO"

Students will present an architecture (HLD + LLD) for a sample application including network, IAM, security, DR, and cost justification.

Hands-on Labs (Module 11)

Lab 73: HLD vs LLD workshop – students document the 3- tier architecture they have deployed.

Lab 74: Build an architecture diagram in draw.io for their final project and attach it to the Wiki.

Lab 75: Resume update – incorporate OCI projects, migrations, hub-spoke, Terraform modules, DR drills.

Lab 76: Conduct a Level 1 mock interview using an AI chat tool; capture questions and answers in personal notes.

Delivery & Evaluation Plan (For Students/Professional)

Mode: Live hands-on plus homework scenarios (Linux basics, automation, migration drills).

Assessment:

- Lab completion and documentation in GitHub Wiki.
- One mid-course project: secure hub-spoke network with Terraform.
- Final capstone: 3 Tier app with DR plan and HLD/LLD pack.